Ecole Nationale des Sciences de l'Informatique

Année Universitaire 2025-2026

Proposition d'un Sujet de Thèse

Titre : **« L'Agent Cognitif de Développement : Conception d'un Système de Question-Réponse Agentique pour l'Assistance Proactive et Contextualisée en Ingénierie Logicielle »**
Directeur de thèse : Pr Moncef Tagina.
Co-directeur de thèse : Pr Wided Lejouad Chaari
Co-encadrement : Dr. Amal Tarifa.
Candidat : Riadh Hasnaoui
Lieu : ENSI

1. Contexte Général

Le développement de logiciels modernes a atteint un niveau de complexité qui met à rude épreuve les capacités cognitives humaines. Les bases de code, constituées de millions de lignes réparties sur des milliers de fichiers, évoluent à un rythme effréné sous l'action collaborative d'équipes souvent distribuées géographiquement. Dans cet écosystème, la tâche du développeur a profondément muté : des études montrent que plus de la moitié de son temps est désormais allouée non pas à la création de code nouveau, mais à des activités de compréhension, telles que la navigation dans le code existant, l'analyse de l'historique des modifications pour en saisir les motivations, ou encore la recherche d'informations pertinentes disséminées entre la documentation, les systèmes de suivi de tickets et les discussions informelles [1]. L'émergence récente des grands modèles de langage (LLMs), incarnée par des outils comme GitHub Copilot, a marqué un tournant en offrant une assistance puissante pour la génération de code. Ces assistants, bien que performants pour des tâches locales et bien définies, fonctionnent principalement sur un mode réactif : ils attendent une instruction explicite pour générer un fragment de code ou répondre à une question factuelle. Leur limitation fondamentale réside dans leur manque de conscience du contexte global du projet. Ils ignorent les décisions architecturales passées, la logique métier implicite, les conventions de l'équipe et les objectifs à long terme, ce qui les cantonne à un rôle d'outil syntaxique avancé plutôt que de véritable partenaire intellectuel. La prochaine rupture technologique dans le domaine de l'IA pour l'ingénierie logicielle consistera à dépasser ce paradigme pour créer des entités capables d'une collaboration cognitive profonde, transformant l'IA d'un simple outil exécutant des ordres en un membre proactif et autonome de l'équipe de développement.

2. Problématique

Le domaine des assistants logiciels pilotés par l'IA est en pleine expansion. Cependant, des études de fond soulignent une tendance dominante [8]: la grande majorité de ces outils, même les plus avancés, opèrent sur un paradigme fondamentalement réactif. Cette observation est parfaitement illustrée par les assistants de code de dernière génération comme GitHub Copilot, basés sur des LLMs tels que GPT-4 [4] ou Code Llama [3]. Leur excellence dans la prédiction du "prochain token" ne leur confère pas une compréhension de l'intention du développeur ni du contexte global du projet. Or, la recherche sur les besoins informationnels des développeurs a démontré de longue date que les questions les plus critiques et chronophages ne sont pas "comment écrire cette boucle ?" mais "pourquoi ce code a-t-il été écrit ainsi ?" ou "quel sera l'impact de mon changement ?" [1, 2].

La recherche actuelle tente activement de combler ce fossé. Des travaux récents [9], proposent par exemple un "assistant de connaissances IA" pour des projets logiciels, marquant une avancée vers une meilleure prise en compte du contexte. Techniquement, de telles approches s'appuient souvent sur des paradigmes comme le Retrieval-Augmented Generation (RAG) [7], qui excellent à retrouver des extraits de documentation textuelle pour ancrer les réponses du modèle. Cependant, le RAG est par nature passif et mal adapté pour interroger des sources de données structurées, exécuter des outils d'analyse de code ou orchestrer des actions complexes, qui sont pourtant essentiels pour un véritable raisonnement en ingénierie logicielle. Cette limitation technologique explique pourquoi même les assistants les plus récents restent principalement des systèmes de présentation d'informations et n'abordent pas encore pleinement la dimension d'agentivité proactive.

En conséquence, la problématique reste entière et se cristallise autour de trois verrous persistants : le paradigme réactif, le manque de synthèse active des connaissances hétérogènes (où le RAG est une première étape nécessaire mais insuffisante), et l'ignorance des processus collaboratifs. Face à ces verrous, la problématique centrale de cette thèse est la suivante : Comment dépasser le modèle de l'outil d'assistance pour concevoir un agent d'IA véritablement cognitif qui utilise le Question-Réponse (QA) non plus comme une fin, mais comme le moteur d'une interaction proactive et collaborative ? Il s'agit de proposer un changement de paradigme vers un "QA Agentique", où l'agent peut raisonner sur le contexte global, prendre l'initiative du dialogue et orchestrer des outils pour devenir un partenaire à part entière dans la résolution de problèmes complexes en ingénierie logicielle [6].

3. Objectifs de la thèse

L'objectif principal de cette thèse est de définir, de développer et de valider un cadre formel pour la création d'agents cognitifs autonomes destinés à l'ingénierie logicielle. Pour ce faire, nous chercherons à doter un agent d'IA de trois capacités fondamentales aujourd'hui absentes des outils existants. Premièrement, nous viserons à développer sa capacité à agir proactivement en modélisant l'intention du développeur à partir de signaux faibles pour initier des interactions pertinentes et non-intrusives. Deuxièmement, nous nous attacherons à lui conférer une capacité de raisonnement sur des connaissances hétérogènes, en lui apprenant à décomposer des problèmes complexes en une série d'appels à des outils spécialisés (analyse de code, interrogation de l'historique Git, etc.) pour construire des réponses informées et fiables. Enfin, le troisième objectif sera d'intégrer cet agent dans les processus **collaboratifs critiques**, en particulier la revue de code, afin qu'il puisse non seulement assister les individus, mais aussi faciliter la communication, la convergence et la qualité au sein de l'équipe.

4. Étapes et Plan de Travail

1. **Étude bibliographique :**
   - Analyse des modèles de représentation des artefacts logiciels et de leurs interdépendances
   - Étude des modèles de représentation du contexte et de l'intention du développeur.
   - Synthèse des modèles de QA et d'agents conversationnels appliqués à l'ingénierie logicielle.
2. **Proposition d'une approche originale :**
   - Développement d'un modèle de **QA Agentique** intégrant une compréhension profonde du contexte du projet et de l'activité du développeur. Ce modèle se caractérise par deux contributions principales :
     - Une **politique de dialogue proactive**, apprise par renforcement, capable de déterminer quand et comment initier une interaction pour maximiser l'aide apportée sans être intrusive.

- Une **architecture de raisonnement et d'action** permettant à l'agent d'interroger de manière autonome des outils et des sources de données hétérogènes (Git, Jira, analyseurs statiques) pour construire ses réponses et ses suggestions.

3. Expérimentation :

- Mise en œuvre du prototype de l'agent sur des corpus de données réelles issues de projets open-source.
- Évaluation de l'agent sur des scénarios d'assistance de complexité variée, couvrant la compréhension de code, l'assistance à la revue de code et l'aide au débogage.
- Comparaison quantitative et qualitative de la performance de l'approche proposée avec des modèles de l'état de l'art.

4. Analyse approfondie :

- Étude de l'impact de la proactivité de l'agent sur des métriques clés comme la charge cognitive du développeur, le temps de complétion des tâches et la qualité du code produit.
- Analyse de la contribution relative de chaque source de connaissance (code, commits, tickets, discussions) à la pertinence et à l'exactitude des interventions de l'agent.
- Etude la possibilité d'une collaboration multi-agents pour créer une proactivité optimisée.

5. Références Bibliographiques

[1] Ko, A. J., DeLine, R., & Venolia, G. (2007). Information needs in collocated software development teams. *Proceedings of the 29th International Conference on Software Engineering*.

[2] LaToza, T. D., & Myers, B. A. (2010). Developers ask reachability questions. *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*.

[3] Nijkamp, E., et al. (2023). Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950*.

[4] OpenAI, et al. (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*.

[5] Vasilescu, B., et al. (2015). Quality and productivity outcomes of peer code review in open-source software. *Proceedings of the 37th International Conference on Software Engineering*.

[6] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Gur, I., & Arivazhagan, N. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

[7] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, *33*, 9459-9474.

[8] Savary-Leblanc, M., Burgueño, L., Cabot, J., Le Pallec, X., & Gérard, S. (2023). Software assistants in software engineering: A systematic mapping study. *Software: Practice and Experience*, *53*(3), 856-892.

[9] Neyem, A., González, L. A., Mendoza, M., Alcocer, J. P. S., Centellas, L., & Paredes, C. (2024). Toward an AI knowledge assistant for context-aware learning experiences in software capstone project development. *IEEE Transactions on Learning Technologies*, *17*, 1599-1614.